

RETO 3 - SEMANA 3 | Examen Final

Unidad 3: Documentación, Evaluación y Comunicación Arquitectónica

Sistema IoT de seguridad perimetral con IA básica

Documento final editable para entrega formal

Este documento cierra el ciclo del proyecto académico y entrega, en un solo artefacto, la documentación final de la arquitectura, la evaluación ATAM simplificada, el diseño de despliegue con Docker Compose y monitoreo, el portafolio GitHub recomendado y el material de comunicación para el Demo Day.

Curso	Arquitectura de Software
Proyecto	Seguridad perimetral IoT con IA básica
Equipo	Jose Barrios - Fabian Alvarez
Entregable	Reto 3 - Documentación, evaluación y comunicación
Formato	DOCX editable y autocontenido
Fecha de edición	Abril 2026

Alcance de esta entrega

La solución mantiene coherencia con el sistema trabajado en los retos previos, pero esta vez se presenta desde una perspectiva final y profesional: vistas de arquitectura, evaluación formal, despliegue reproducible y material de defensa oral.

0. Resumen ejecutivo

La arquitectura final se apoya en una cadena operativa simple y defendible: un dispositivo edge o un simulador publica eventos de movimiento al broker MQTT; Node-RED actúa como gateway táctico, normaliza los mensajes, aplica una IA básica explicable, decide la severidad, persiste evidencia y actualiza el tablero; el operador consulta estado, historial y disponibilidad desde una interfaz web.

El valor de esta versión final no está solo en el flujo funcional, sino en su capacidad de ser comunicada y evaluada. Por eso el documento emplea el enfoque Views & Beyond con tres vistas complementarias, utiliza un ATAM simplificado para analizar riesgos y puntos sensibles, y aterriza el despliegue en contenedores con verificaciones operativas mínimas.

El resultado es una entrega autocontenida y lista para revisión académica: explica la arquitectura, muestra cómo se desplegaría, documenta decisiones y deja preparado el guion para una defensa de 10 a 12 minutos con evidencia observable.

0.1 Matriz de trazabilidad frente a la rúbrica

#	Criterio del Reto 3	Evidencia dentro del documento	Estado
1	Documentación completa (Views & Beyond)	Sección 1; Figuras 1, 2 y 3; Tablas 1, 2, 3 y 4	Cubierto
2	Evaluación de arquitectura (ATAM simplificado)	Sección 2; Tablas 5, 6, 7, 8 y 9	Cubierto
3	Docker Compose + monitoreo	Sección 3; Tablas 10, 11 y 12; Anexos A y B	Cubierto
4	Portafolio GitHub completo	Sección 4; Tablas 13, 14 y 15	Cubierto
5	Demo Day + póster/video	Sección 5; Tablas 16, 17, 18 y 19	Cubierto

Lectura recomendada

Para una revisión técnica rápida conviene leer la Sección 1 (vistas), pasar a la Sección 2 (ATAM) y terminar en la Sección 3 (despliegue). Las Secciones 4 y 5 convierten esa base técnica en un portafolio y en un discurso de defensa.

1. Documentación completa (Views & Beyond)

El modelo C4 ya había permitido representar el contexto, los contenedores y los componentes principales del sistema. En esta etapa final se documenta la arquitectura con una mirada Views & Beyond, es decir, con vistas que responden a preocupaciones distintas de distintos interesados.

La vista de módulos responde a cómo se organiza el software y cómo se reparte la responsabilidad; la vista C&C responde a cómo se comporta la solución en tiempo de ejecución; y la vista de deployment responde a dónde se ejecuta cada pieza y cómo se sostiene operativamente el sistema.

1.1 Vista de módulos

La vista de módulos divide el sistema en unidades estables de responsabilidad. Su objetivo principal es mejorar modificabilidad, legibilidad del diseño y trazabilidad entre requisitos de calidad y estructura del software.

Vista de módulos

Descomposición estática del sistema IoT en paquetes y módulos con responsabilidades claras.

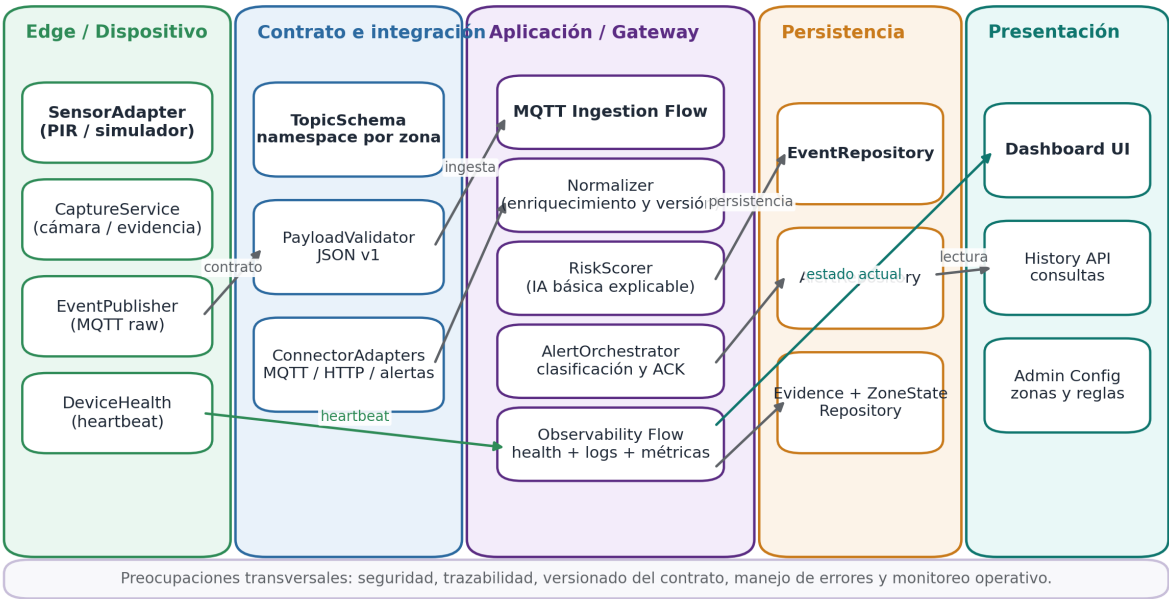


Figura 1. Vista de módulos del sistema IoT de seguridad perimetral.

La figura muestra cinco agrupaciones principales: dispositivo edge, contrato e integración, gateway de aplicación, persistencia y presentación. Aunque Node-RED sigue siendo el centro táctico del prototipo, la arquitectura evita convertirlo en una caja negra: se separan explícitamente ingesta, normalización, score, orquestación, repositorios y observabilidad.

ID	Módulo	Responsabilidad principal	Interfaces	Drivers impactados
M1	Edge Adapter	Captura movimiento o simula eventos; encapsula el contacto con PIR/ESP32 y publica al bus.	Publicación MQTT en tópicos raw; heartbeat del dispositivo.	Latencia, costo y viabilidad.
M2	Gestor de contrato	Versiona tópicos y payloads; valida campos mínimos y permite evolución controlada.	Esquema JSON v1; namespaces por zona y tipo de evento.	Modificabilidad e interoperabilidad.
M3	Ingesta y	Recibe mensajes del broker, convierte a	Suscripción MQTT;	Modificabilidad y

ID	Módulo	Responsabilidad principal	Interfaces	Drivers impactados
	normalización	objeto interno y agrega metadatos operativos.	objeto de evento interno.	trazabilidad.
M4	RiskScorer	Aplica IA básica explicable para reducir falsos positivos con una regla reproducible.	Función de score sobre señales de movimiento, horario y confianza.	Usabilidad operativa y calidad de alerta.
M5	AlertOrchestrator	Clasifica el evento, dispara notificaciones, gestiona ACK y actualiza el estado de zona.	Publicación de alertas; comandos de reconocimiento.	Respuesta operativa y trazabilidad.
M6	Capa Repository	Encapsula persistencia de eventos, alertas, evidencia y estado actual.	Operaciones de alto nivel sobre PostgreSQL.	Mantenibilidad y consistencia.
M7	Dashboard/API	Expone estado actual, historial, evidencia y acciones del operador.	REST/HTTP y vista web del tablero.	Usabilidad y seguridad.
M8	Observabilidad	Concentra health checks, heartbeats, logs y métricas de disponibilidad.	Endpoints /health y /healthz; monitor externo.	Disponibilidad y operación.

Decisiones arquitectónicas de la vista de módulos

ID	Decisión	Razonamiento
DM-01	Mantener límites explícitos entre captura, contrato, orquestación y persistencia.	Reduce acoplamiento y facilita cambios en firmware o UI sin reescribir la lógica central.
DM-02	Tratar el contrato MQTT como un módulo independiente y versionado.	Permite crecer a nuevas zonas o sensores sin romper consumidores existentes.
DM-03	Concentrar la IA básica en un módulo aislado del Gateway.	La heurística puede reemplazarse por otra técnica sin afectar el resto del flujo.
DM-04	Incluir observabilidad como módulo transversal desde el diseño.	Evita que la operación quede como una tarea posterior y mejora diagnósticos en demo y despliegue.

Síntesis de la vista

Esta vista privilegia la modificabilidad: el edge queda liviano, el contrato se controla aparte, la lógica de negocio se orquesta en el gateway y la persistencia se encapsula con repositorios.

1.2 Vista C&C (componentes y conectores)

La vista C&C hace visible el comportamiento del sistema en ejecución. Es la vista que mejor explica el flujo completo del prototipo porque representa los componentes activos y los conectores concretos que permiten mover eventos, consultar datos y observar disponibilidad.

Vista C&C (componentes y conectores)

Vista dinámica de ejecución: flujo de datos, conectores de comunicación y observabilidad operativa.

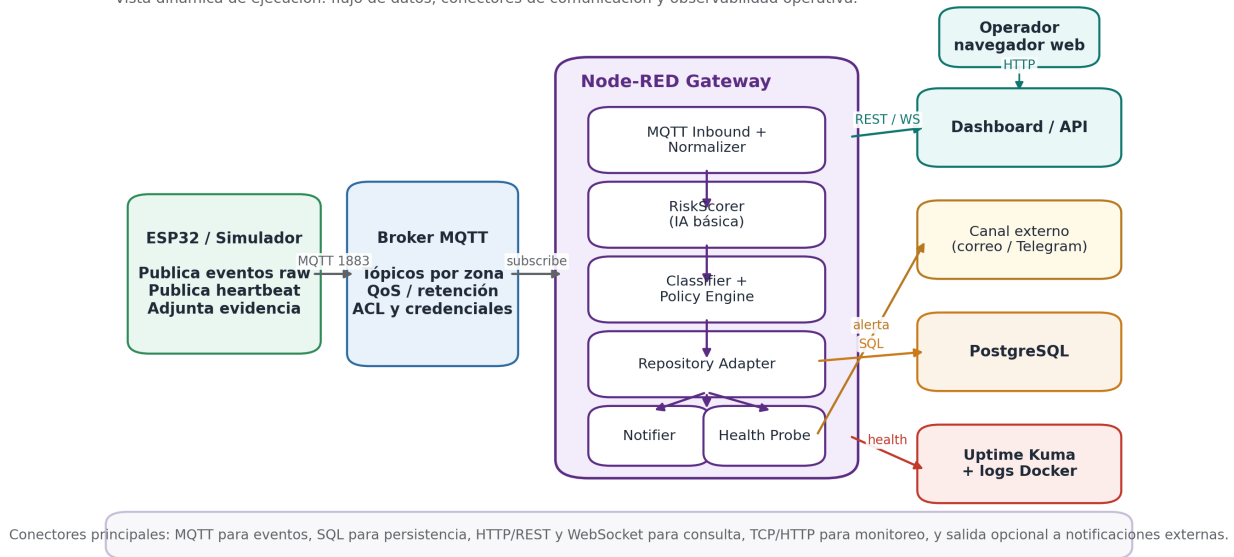


Figura 2. Vista C&C con componentes de ejecución y conectores principales.

La decisión clave aquí es separar el bus de eventos del plano de consultas. MQTT atiende ingestión y propagación de estado; la API atiende lectura del tablero; y PostgreSQL concentra la verdad persistente del sistema. Con ello se evita usar una sola tecnología para todas las responsabilidades y se preserva claridad táctica.

Tipo	Elemento	Responsabilidad	Conector / protocolo	Observación
Componente	Publicador edge	Genera eventos raw y heartbeats desde hardware real o simulación.	MQTT/TCP	Mantiene al dispositivo simple y desacoplado.
Componente	Broker MQTT	Distribuye eventos por tópicos, aplica ACL y soporta desacoplamiento temporal.	MQTT/TCP	Es la espina dorsal de la comunicación Pub-Sub.
Componente	MQTT Inbound + Normalizer	Recibe, valida y estandariza mensajes entrantes dentro de Node-RED.	MQTT interno	Protege a la lógica de negocio de variaciones del payload.
Componente	RiskScorer	Calcula score final y confianza operativa del evento.	Invocación interna	Encapsula la IA básica explicable.
Componente	Classifier + Policy Engine	Decide si el evento se descarta, queda como sospechoso o se promueve a alerta.	Invocación interna	Materializa reglas tácticas del dominio.
Componente	Repository Adapter	Persistencia de eventos, alertas, evidencia y cambios de estado.	SQL	Oculto detalles de almacenamiento.
Componente	Dashboard/API	Entrega estado actual, historial y evidencia al operador.	HTTP/REST y WebSocket	Vista orientada a lectura y seguimiento.
Componente	Notifier	Envía alertas a correo o Telegram cuando	HTTP/SMTP/API	Es opcional y no bloquea

Tipo	Elemento	Responsabilidad	Conector / protocolo	Observación
e		la política lo exige.	externa	el pipeline principal.
Componente	Health Probe / Uptime	Verifica disponibilidad de servicios y centraliza evidencia operativa.	HTTP/TCP	Mide salud y tiempo de recuperación.
Conector	Conectores principales	MQTT entre edge y broker; HTTP/REST con el dashboard; SQL hacia PostgreSQL; TCP/HTTP para monitoreo.	Mixto	Cada conector se elige por adecuación al uso concreto.

Decisiones arquitectónicas de la vista C&C

ID	Decisión	Razonamiento
DC-01	MQTT se usa solo para eventos y estados; no para consultas históricas.	Evita sobrecargar el broker con responsabilidades de consulta y preserva separación de intereses.
DC-02	El dashboard consume una API y no consulta el broker ni la base de datos directamente.	Reduce acoplamiento, mejora seguridad y deja espacio para controles de acceso y caching.
DC-03	Las notificaciones son asíncronas y opcionales respecto del flujo principal.	Un fallo en Telegram o correo no debe impedir que la alerta se registre y sea visible en el tablero.
DC-04	Los componentes de salud se conectan por endpoints explícitos.	Hace verificable la disponibilidad del sistema y permite reinicios automáticos basados en evidencia.

1.3 Vista de deployment

La vista de deployment traduce la arquitectura a nodos de ejecución. En esta etapa se adopta un despliegue reproducible con Docker Compose sobre un único host de laboratorio, más un nodo edge físico o simulado. Esta decisión es coherente con el alcance académico: suficiente realismo operativo sin complejidad de infraestructura innecesaria.

Vista de deployment

Asignación de software a nodos físicos y contenedores con health checks, logs y monitoreo.

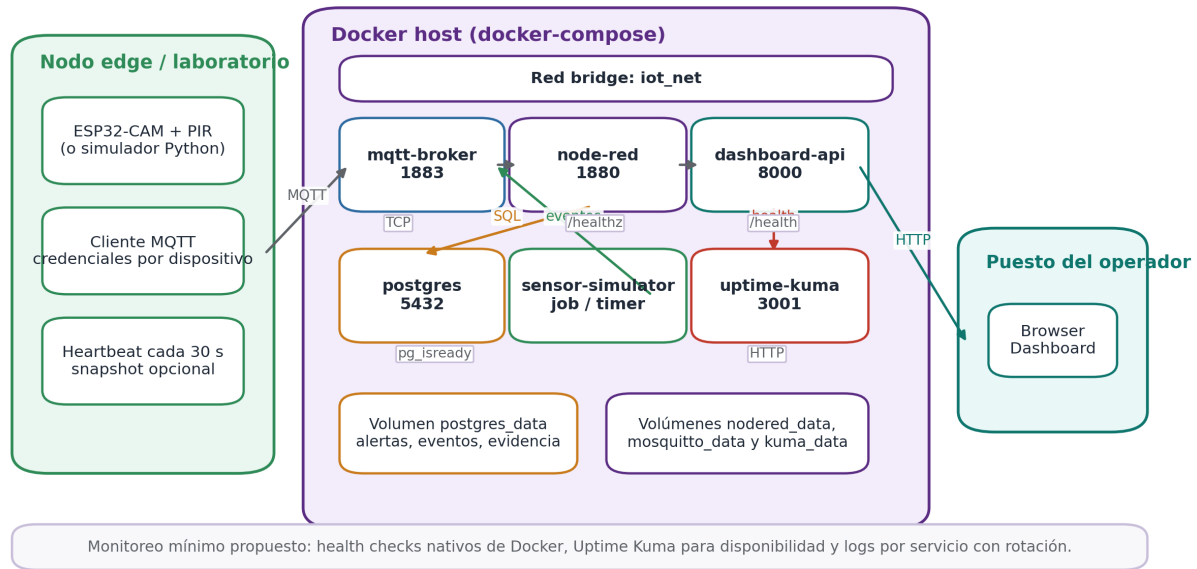


Figura 3. Vista de deployment con nodos, contenedores, volúmenes y monitoreo.

Tipo	Elemento	Responsabilidad	Ubicación	Puerto	Health check
Nodo / artefacto	Sensor real o simulador	Emite eventos y heartbeats usando el contrato MQTT del sistema.	Laboratorio o red local	Salida MQTT 1883	Heartbeat por tópico y estado del proceso.
Contenedor	mqtt-broker	Mensajería desacoplada con tópicos por zona y control de acceso.	Docker host	1883	Suscripción a tópico del sistema y puerto TCP abierto.
Contenedor	node-red	Gateway del sistema; ingesta, reglas, score, persistencia y alarmas.	Docker host	1880	GET /healthz y reinicio automático.
Contenedor	dashboard-api	Tablero web, API de consulta y operaciones de ACK.	Docker host	8000	GET /health y respuesta HTTP 200.
Contenedor	postgres	Persistencia de eventos, alertas, evidencia y estado de zona.	Docker host	5432	pg_isready y volumen persistente.
Contenedor	uptime-kuma	Monitoreo mínimo de disponibilidad y tiempos de respuesta.	Docker host	3001	GET / y monitores activos.
Volúmenes	postgres_data,	Persistencia de estado y recuperación	Docker host	N/A	Datos

Tipo	Elemento	Responsabilidad	Ubicación	Puerto	Health check
	nodered_data, mosquitto_data, kuma_data	tras reinicios.			conservados entre reinicios.

Decisiones arquitectónicas de la vista de deployment

ID	Decisión	Razonamiento
DD-01	Se consolida el despliegue en un solo Docker host de laboratorio.	Minimiza complejidad operativa en el entorno académico y hace reproducible la demo.
DD-02	La persistencia pasa a PostgreSQL para la etapa final.	Aumenta el costo operativo respecto de SQLite, pero mejora consistencia, trazabilidad y contenedorización.
DD-03	Se añade Uptime Kuma como monitoreo mínimo de disponibilidad.	Aporta visibilidad operacional suficiente sin introducir una plataforma observability demasiado pesada.
DD-04	Todos los servicios críticos arrancan con restart policy y health checks.	La recuperación tras fallos deja de depender de intervención manual continua.

1.4 Matriz de correspondencia entre vistas

La siguiente matriz une el mundo estático, el runtime y el despliegue. Su propósito es evitar que las vistas queden aisladas: cada módulo debe poder rastrearse hasta un componente ejecutable y hasta un artefacto concreto de despliegue.

Vista de módulos	Vista C&C	Artefacto de deployment
Edge Adapter	Publicador edge	Sensor real / simulador
Gestor de contrato	MQTT Inbound + Normalizer	node-red + definición de tópicos
RiskScorer	RiskScorer	node-red
AlertOrchestrator	Classifier + Policy Engine + Notifier	node-red
Capa Repository	Repository Adapter	node-red + postgres
Dashboard/API	Dashboard/API	dashboard-api
Observabilidad	Health Probe + Uptime	node-red + uptime-kuma + Docker health checks

2. Evaluación de arquitectura (ATAM simplificado)

El propósito de esta evaluación no es declarar la arquitectura perfecta, sino hacer explícitos sus riesgos y confirmar si las decisiones tomadas responden a los drivers priorizados. Se adopta un ATAM simplificado porque permite contrastar atributos de calidad, escenarios, riesgos, puntos sensibles y trade-offs sin exigir una dinámica formal de taller.

La evaluación se estructura en cinco pasos: revisar drivers, priorizar escenarios, aplicar un checklist de consistencia, identificar riesgos y sensibilidad, y cerrar con trade-offs y juicio global sobre la aceptabilidad del diseño.

2.1 Drivers y tácticas observables

ID	Driver	Prioridad	Táctica arquitectónica observable
D1	Seguridad	Alta	ACL en MQTT, credenciales por dispositivo, API separada y exposición controlada de puertos.
D2	Disponibilidad	Alta	Restart policies, health checks, heartbeats, monitoreo externo y volúmenes persistentes.
D3	Rendimiento / latencia	Alta	Pipeline orientado a eventos; procesamiento liviano; separación de consulta e ingestión.
D4	Escalabilidad	Media-Alta	Tópicos por zona, contratos versionados, módulos desacoplados y dashboard orientado a lectura.
D5	Modificabilidad	Alta	Node-RED como gateway táctico, repositorios, contratos explícitos y separación por módulos.
D6	Trazabilidad	Media-Alta	Persistencia transaccional de eventos y alertas, evidencia asociada y ACK del operador.
D7	Costo / viabilidad	Media	Stack open source, contenedores ligeros y monitoreo mínimo en lugar de plataforma enterprise.
D8	Usabilidad operativa	Media	Dashboard con estado actual, historial, evidencia y severidad comprensible.

2.2 Escenarios priorizados en la evaluación

ID	Atributo	Escenario	Valor negocio	Riesgo técnico	Prioridad
S1	Seguridad	Dispositivo no registrado intenta publicar en un tópico operativo.	Alta	Media	Alta
S2	Disponibilidad	El broker o Node-RED se reinicia mientras llegan eventos.	Alta	Alta	Alta
S3	Latencia	Una intrusión válida debe verse en el dashboard en menos de 2 segundos.	Alta	Media	Alta
S4	Modificabilidad	Se agrega una nueva zona o un sensor magnético sin rehacer el flujo existente.	Media-Alta	Media	Alta
S5	Trazabilidad	El operador debe reconstruir la historia de una alerta con evidencia y ACK.	Alta	Media	Alta

ID	Atributo	Escenario	Valor negocio	Riesgo técnico	Prioridad
S6	Operabilidad	Un servicio cae y debe detectarse y reportarse en menos de un minuto.	Media	Media	Media-Alta

2.3 Checklist ATAM simplificado

ID	Pregunta	Resultado	Observación
AT-01	¿La arquitectura responde a los drivers priorizados?	Sí	Los drivers D1-D8 tienen tácticas explícitas y trazables.
AT-02	¿Existen contratos claros entre productores y consumidores?	Sí	El esquema JSON v1 y los namespaces por tópico reducen ambigüedad.
AT-03	¿Se aislaron responsabilidades de negocio e infraestructura?	Sí	Gateway, dashboard y repositorios mantienen separación razonable.
AT-04	¿Hay puntos únicos de falla identificados?	Sí	Broker, Node-RED y PostgreSQL siguen siendo SPOF del prototipo.
AT-05	¿La solución puede observarse operativamente?	Parcial	Hay health checks y uptime, pero no tracing distribuido ni métricas avanzadas.
AT-06	¿La seguridad es suficiente para un piloto académico?	Parcial	Es adecuada para laboratorio; para producción convendría TLS y gestión de secretos.
AT-07	¿La arquitectura admite crecimiento moderado?	Sí	Permite más zonas y más consumidores sin tocar el edge.
AT-08	¿La persistencia protege trazabilidad y consulta histórica?	Sí	Se almacenan eventos, alertas, evidencia y ACK de atención.
AT-09	¿El despliegue es reproducible?	Sí	Docker Compose define arranque consistente en laboratorio o demo.
AT-10	¿El costo operativo sigue siendo razonable?	Sí	La plataforma usa componentes open source y monitoreo ligero.

2.4 Riesgos arquitectónicos identificados

ID	Riesgo	Impacto	Severidad	Prob.	Mitigación
R1	Broker MQTT como punto único de falla	Una caída del broker detiene la ingestión de eventos.	Alta	Media	Replica del broker o backup warm standby; persistencia y restart policy.
R2	Node-RED centraliza demasiada lógica	Un error en el gateway impacta clasificación, persistencia y notificación.	Alta	Media	Separar subflujos, pruebas por flujo y eventual partición por dominios.
R3	Heurística de IA básica insuficiente en entornos ruidosos	Aumentan falsos positivos o falsos negativos.	Media-Alta	Alta	Ajuste de umbral, fusión con sensor magnético o análisis de imagen posterior.
R4	Credenciales en variables de entorno de laboratorio	Riesgo de exposición accidental en repositorio o en la máquina de demo.	Alta	Media	Usar .env no versionado, Docker secrets o gestor de secretos en una siguiente fase.
R5	Crecimiento del almacenamiento de evidencia	El disco local puede agotarse y afectar disponibilidad.	Media	Media	Política de retención, compresión y offload a almacenamiento externo.
R6	Monitoreo mínimo no detecta métricas funcionales profundas	Se detecta caída, pero no todas las degradaciones de negocio.	Media	Media	Agregar métricas de latencia y trazas de eventos end-to-end.

2.5 Puntos sensibles

ID	Parámetro sensible	Descripción	Atributos afectados
SP-01	Umbral de score de alerta	Cambiar 0.70 a 0.55 aumenta detección, pero también falsos positivos.	Usabilidad, seguridad y carga operativa.
SP-02	QoS MQTT	QoS 0 reduce latencia; QoS 1 mejora entrega pero puede duplicar mensajes.	Latencia, disponibilidad y consistencia.
SP-03	Intervalo de heartbeat	Heartbeats muy frecuentes consumen recursos; muy espaciados retrasan detección de falla.	Disponibilidad y costo operativo.
SP-04	Política de retención de evidencia	Más retención mejora auditoría, pero aumenta almacenamiento y costo.	Trazabilidad y viabilidad.
SP-05	Timeout de notificación externa	Esperar demasiado bloquea recursos; un timeout corto puede perder avisos.	Disponibilidad y respuesta operativa.
SP-06	Tamaño de la base y estrategia de índices	Consultas históricas sin índice por zona/fecha degradan el dashboard.	Usabilidad y rendimiento.

2.6 Trade-offs documentados

ID	Decisión	Qué se gana	Qué se sacrifica
TO-01	Node-RED como gateway central	Modificabilidad, velocidad de cambio y facilidad de demo.	Mayor dependencia de un componente central y posible SPOF.
TO-02	PostgreSQL en lugar de SQLite	Trazabilidad más sólida, mejor concurrencia y despliegue contenedorizado.	Más consumo de recursos y administración adicional.
TO-03	Monitoreo ligero con Uptime Kuma	Visibilidad rápida y simple de disponibilidad.	No entrega el nivel de detalle de Prometheus/Grafana o tracing distribuido.
TO-04	Dashboard orientado a lectura con API separada	Mejora seguridad, claridad y desacoplamiento.	Requiere una capa adicional respecto de consultar la base directamente.

Resultado de la evaluación

La arquitectura es adecuada para un prototipo funcional y una defensa académica robusta. Cumple bien en modificabilidad, trazabilidad y despliegue reproducible, pero sigue siendo una arquitectura de laboratorio: para un entorno crítico requeriría alta disponibilidad real, secretos gestionados y observabilidad más profunda.

3. Despliegue con Docker Compose + monitoreo

El objetivo operativo de esta sección es transformar la arquitectura en una composición reproducible. Docker Compose se adopta porque permite describir servicios, dependencias, volúmenes, variables de entorno y health checks en un solo archivo, facilitando arranque y demostración.

El monitoreo propuesto es deliberadamente liviano: health checks nativos de Docker, logs rotados por servicio y Uptime Kuma para disponibilidad y tiempos de respuesta. No pretende competir con una plataforma enterprise, sino cubrir de manera suficiente la necesidad del curso: servicios contenerizados, visibles y verificables.

3.1 Catálogo de servicios

Servicio	Imagen	Responsabilidad	Puerto	Volumen	Verificación
mqtt-broker	eclipse-mosquitto:2	Bus de mensajería para eventos y estados.	1883	mosquitto_data	Suscripción local a tópico del sistema.
node-red	nodered/node-red:3.1	Gateway del sistema, reglas, score y orquestación.	1880	nodered_data	GET /healthz => 200.
dashboard-api	Imagen propia (FastAPI/Flask)	Dashboard web, historial y ACK del operador.	8000	opcional cache	GET /health => 200.
postgres	postgres:16-alpine	Persistencia del dominio operacional.	5432	postgres_data	pg_isready.
uptime-kuma	louislam/uptime-kuma:1	Monitoreo mínimo de disponibilidad.	3001	kuma_data	GET / => 200.
sensor-simulator	Imagen propia Python	Publicador controlado para demo o pruebas repetibles.	N/A	sin persistencia	Heartbeat visible en Node-RED.

3.2 Archivo docker-compose propuesto

El archivo completo del despliegue se presenta en el Anexo A. Desde el punto de vista arquitectónico, la composición tiene cuatro rasgos valiosos: dependencias explícitas con condición healthy, volúmenes persistentes, restart policy homogénea y un stack de monitoreo suficientemente simple para ser explicado en clase.

Criterio operativo

El despliegue debe considerarse aceptable cuando broker, Node-RED, dashboard y PostgreSQL pasan a estado healthy; el simulador puede publicar un evento; y el operador puede verlo en el dashboard y en la vista de monitoreo.

3.3 Health checks y recuperación

Servicio	Chequeo	Criterio de éxito	Acción esperada
mqtt-broker	Tópico de sistema o puerto TCP 1883	Broker accesible y respondiendo a suscripciones.	Reinicio automático del contenedor y alerta en Uptime Kuma.
node-red	GET /healthz	HTTP 200 con estado de MQTT y DB.	Reinicio automático; el monitor lo marca como degradado.

Servicio	Chequeo	Criterio de éxito	Acción esperada
dashboard-api	GET /health	HTTP 200 y acceso a base de datos.	Reinicio automático y marca de indisponibilidad.
postgres	pg_isready	Base aceptando conexiones del usuario de la aplicación.	Node-RED y dashboard esperan hasta que la base esté healthy.
uptime-kuma	GET /	Interfaz y monitores disponibles.	Verificación manual y reinicio automático si aplica.

3.4 Logs y métricas de disponibilidad

Métrica o evidencia	Fuente	Uso arquitectónico
Disponibilidad por servicio	Uptime Kuma	Porcentaje de uptime de broker, Node-RED y dashboard.
Tiempo de respuesta del dashboard	Uptime Kuma / health HTTP	Mide degradación del servicio expuesto al operador.
Último heartbeat recibido	Node-RED + dashboard	Indica si el edge está vivo aunque no haya eventos.
Latencia evento -> alerta	Diferencia entre timestamps en PostgreSQL	Permite validar el objetivo operativo de punta a punta.
Incidentes de reinicio	docker compose ps / logs	Evidencia básica de estabilidad y recuperación.

3.5 Protocolo de verificación

Paso	Acción	Comando / medio	Evidencia esperada
1	Levantar la composición	docker compose up -d --build	Todos los contenedores críticos pasan a estado healthy.
2	Verificar estado de servicios	docker compose ps	mqtt-broker, node-red, dashboard-api, postgres y uptime-kuma activos.
3	Inspeccionar monitoreo	Abrir http://localhost:3001	Monitores del broker, dashboard y Node-RED en verde.
4	Publicar evento de prueba	Ejecutar el simulador o publicar por MQTT	Node-RED recibe el evento y calcula score.
5	Validar persistencia	Consultar dashboard o base	Se registra el evento y, si aplica, la alerta.
6	Validar operación del operador	Abrir http://localhost:8000	El tablero muestra estado actual, historial y ACK.

Decisión de alcance

No se incorpora orquestación avanzada, balanceo ni HA real porque el entorno objetivo es un laboratorio académico. Sin embargo, el archivo compose deja preparada una base ordenada para evolucionar hacia una topología mayor.

4. Portafolio GitHub completo

El portafolio GitHub no se limita a alojar código; debe contar la historia técnica del sistema y permitir que otra persona lo reconstruya y lo evalúe. Por eso se propone una estructura en la que conviven artefactos de ejecución, documentación arquitectónica, decisiones ADR y material de demostración.

Una buena señal de madurez del repositorio es que un evaluador pueda abrir el README, entender el problema, ver la arquitectura, seguir instrucciones de instalación y ubicar las evidencias sin pedir contexto adicional.

4.1 Estructura recomendada del repositorio

```
/
|-- README.md
|-- docker-compose.yml
|-- .env.example
|-- docs/
|   |-- c4/
|   |-- views/
|   |-- adr/
|   |-- atam/
|   |-- demo/
|-- mosquito/
|   |-- mosquito.conf
|   |-- passwords
|-- nodered/
|   |-- flows.json
|   |-- settings.js
|-- dashboard/
|   |-- Dockerfile
|   |-- requirements.txt
|   |-- app/
|-- simulator/
|   |-- Dockerfile
|   |-- publisher.py
|-- db/
|   |-- init.sql
```

Área	Contenido	Propósito
Raíz	README.md, docker-compose.yml, .env.example, LICENSE.	Entrada principal del proyecto y guía de arranque.
docs/c4	Diagramas C4 niveles 1, 2 y 3.	Trazabilidad entre diseño inicial, táctico y final.
docs/views	Vista de módulos, C&C y deployment.	Material del Reto 3 y vistas Views & Beyond.
docs/adr	ADR-001 a ADR-004.	Decisiones arquitectónicas versionadas.
docs/atam	Checklist, riesgos, puntos sensibles y trade-offs.	Evidencia de evaluación final.
docs/demo	Guion de presentación, póster y video.	Material listo para Demo Day.
nodered / mosquito / dashboard / simulator / db	Código y configuración operativa.	Artefactos ejecutables del prototipo.

4.2 README profesional propuesto

Sección	Contenido esperado
1. Descripción del proyecto	Problema, alcance del sistema IoT y objetivo operacional.
2. Arquitectura	Resumen de patrones, C4 y vistas finales; incluir al menos una imagen por nivel.
3. Stack tecnológico	MQTT, Node-RED, dashboard/API, PostgreSQL, Docker Compose y monitoreo.
4. Requisitos previos	Docker, Docker Compose, puertos requeridos y variables de entorno.
5. Instalación rápida	Clonar, configurar .env, levantar servicios y validar health checks.
6. Flujo de demo	Cómo publicar un evento y cómo comprobarlo en el dashboard y monitoreo.
7. ADR y evaluación	Enlaces a decisiones arquitectónicas y al resumen ATAM.
8. Evidencia	Screenshots, video demo, enlace del póster o video de 2 minutos.

Fragmento base para iniciar el README

```
# Sistema IoT de Seguridad Perimetral con IA básica

## Descripción
Prototipo IoT para vigilancia perimetral con eventos MQTT,
orquestración en Node-RED, dashboard web, persistencia en
PostgreSQL y monitoreo mínimo de disponibilidad.

## Arranque rápido
1. Copiar `.env.example` a `.env`
2. Configurar credenciales y zona por defecto
3. Ejecutar `docker compose up -d --build`
4. Verificar health checks con `docker compose ps`
5. Abrir:
   - Dashboard: http://localhost:8000
   - Node-RED: http://localhost:1880
   - Uptime Kuma: http://localhost:3001

## Demo básica
Publicar un evento de movimiento y validar:
- recepción en Node-RED,
- persistencia en PostgreSQL,
- visualización en dashboard,
- disponibilidad visible en monitoreo.
```

4.3 ADR que deben versionarse

ID	Decisión	Motivación	Consecuencia principal
ADR-001	Usar MQTT + Pub-Sub como backbone de eventos	Desacoplar productores y consumidores del sistema perimetral.	Escalabilidad y flexibilidad; dependencia del broker como SPOF.
ADR-002	Concentrar orquestación e IA básica en Node-RED	Reducir cambios en firmware y acelerar iteración del prototipo.	Alta modificabilidad; mayor criticidad del gateway.
ADR-003	Persistir en PostgreSQL mediante repositorios	Garantizar trazabilidad y consultas históricas confiables.	Mejor consistencia; mayor huella operativa que SQLite.
ADR-004	Contenerizar con Docker Compose y monitorear con health checks + Uptime Kuma	Reproducibilidad y observabilidad de laboratorio.	Mejor operación; monitoreo menos profundo que una plataforma enterprise.

ADR-001 | Usar MQTT + Pub-Sub como backbone de eventos

Contexto: Desacoplar productores y consumidores del sistema perimetral. Consecuencia: Escalabilidad y flexibilidad; dependencia del broker como SPOF.

ADR-002 | Concentrar orquestación e IA básica en Node-RED

Contexto: Reducir cambios en firmware y acelerar iteración del prototipo. Consecuencia: Alta modificabilidad; mayor criticidad del gateway.

ADR-003 | Persistir en PostgreSQL mediante repositorios

Contexto: Garantizar trazabilidad y consultas históricas confiables. Consecuencia: Mejor consistencia; mayor huella operativa que SQLite.

ADR-004 | Contenerizar con Docker Compose y monitorear con health checks + Uptime Kuma

Contexto: Reproducibilidad y observabilidad de laboratorio. Consecuencia: Mejor operación; monitoreo menos profundo que una plataforma enterprise.

4.4 Evidencia mínima que debe contener el repositorio

Tipo de evidencia	Contenido mínimo
Código y configuración	docker-compose.yml, Dockerfiles, flows.json, mosquitto.conf, init.sql.
Documentación	README, vistas finales, ATAM simplificado, ADRs y decisiones tácticas.
Evidencia visual	Capturas del dashboard, Uptime Kuma, flujo Node-RED y terminal MQTT.
Video de funcionamiento	Demo de 5 min o Demo Day según formato del curso.
Artefacto de comunicación	Poster académico o video resumen de 2 min.

5. Demo Day: presentación 10-12 min + póster/video 2 min

La defensa oral debe mostrar que la arquitectura no es solo un dibujo elegante, sino una solución que se puede explicar, evaluar y operar. Por ello la presentación debe alternar tres tipos de evidencia: vistas arquitectónicas, razonamiento de calidad y funcionamiento observable del prototipo.

A diferencia de una demo técnica improvisada, el Demo Day exige un relato claro: problema, decisiones, trade-offs, despliegue y evidencia. El objetivo final es que el jurado entienda por qué la arquitectura propuesta es razonable para el contexto del proyecto.

5.1 Guion de presentación de 10 a 12 minutos

Tiempo	Bloque	Mensaje central	Evidencia
0:00 - 0:45	Problema y objetivo	Qué problema resuelve el sistema y por qué importa.	Portada + caso de uso realista.
0:45 - 2:00	Evolución arquitectónica	Cómo se pasó de diseño inicial a arquitectura táctica y final.	C4 niveles 1-3 y vistas finales.
2:00 - 3:30	Vista de módulos	Qué responsabilidades existen y cómo se separan.	Figura 1 + tabla de módulos.
3:30 - 5:00	Vista C&C	Cómo corre el sistema en tiempo real y qué conectores usa.	Figura 2 + contrato MQTT.
5:00 - 6:30	Deployment y operación	Cómo se despliega con Docker Compose y cómo se monitorea.	Figura 3 + health checks.
6:30 - 8:00	Evaluación ATAM	Qué riesgos siguen presentes y qué trade-offs se aceptaron.	Riesgos, puntos sensibles y trade-offs.
8:00 - 10:00	Demo en vivo	Publicar evento, ver alerta, consultar historial y revisar monitoreo.	Terminal MQTT + dashboard + Uptime Kuma.
10:00 - 11:30	Cierre y roadmap	Qué se logró, qué haría falta para producción y por qué la arquitectura es defendible.	Conclusión + próximos pasos.

5.2 Contenido sugerido para el póster académico

Bloque	Mensaje	Apoyo visual
Problema	Vigilancia perimetral de bajo costo con evidencia y alerta útil.	Una frase clara y una imagen del flujo.
Arquitectura	MQTT + Node-RED Gateway + Dashboard/API + PostgreSQL.	Mini diagrama C4 o deployment.
Patrones y decisiones	Pub-Sub, Gateway, Repository, Docker Compose, monitoreo.	Tabla corta de decisiones.
Evaluación	Riesgos, puntos sensibles y trade-offs aceptados.	Matriz pequeña con semáforos o prioridades.
Resultados	Flujo de punta a punta, health checks, trazabilidad y demo.	Capturas del dashboard y Uptime Kuma.
Conclusión	Arquitectura defendible, operable y lista para crecer.	Mensaje final de valor.

5.3 Guion del video resumen de 2 minutos

Escena	Duración	Narrativa	Visual
Escena 1	0-20 s	Presentar el problema y el objetivo del sistema.	Portada + imagen de arquitectura general.
Escena 2	20-45 s	Mostrar el flujo edge -> MQTT -> Node-RED -> dashboard.	Animación simple o captura del prototipo.
Escena 3	45-70 s	Explicar los patrones y por qué reducen acoplamiento.	Resaltar broker, gateway y repositorios.
Escena 4	70-95 s	Mostrar el despliegue con Docker Compose y monitoreo.	Captura de Uptime Kuma y estado de contenedores.
Escena 5	95-120 s	Cerrar con riesgos controlados y valor del proyecto.	Resumen visual + enlace al repositorio.

5.4 Preguntas probables y líneas de defensa

Pregunta probable	Respuesta recomendada
¿Por qué Node-RED y no microservicios puros desde el inicio?	Porque el objetivo del curso es maximizar claridad arquitectónica y velocidad de iteración con bajo costo. Node-RED actúa como gateway táctico y deja abierta una futura partición en servicios.
¿Cuál es el principal riesgo actual?	El punto único de falla en broker, gateway y base dentro del mismo host. Es aceptable para laboratorio, pero el siguiente paso sería alta disponibilidad y separación física.
¿La IA básica es realmente IA?	Es una heurística explicable y reproducible. En el alcance académico funciona como validación inteligente básica; la arquitectura permite reemplazarla por un modelo más sofisticado.
¿Por qué PostgreSQL al final si antes se consideró SQLite?	Porque la fase final exige despliegue y trazabilidad más sólidos. PostgreSQL mejora concurrencia, consulta histórica y contenedorización.
¿Cómo se evita que una falla de notificación rompa el sistema?	Las notificaciones son asíncronas y no bloquean el pipeline principal. La alerta se registra y aparece en el dashboard aunque falle Telegram o correo.
¿Qué evidencia demuestra observabilidad?	Health checks, uptime por servicio, logs básicos y heartbeats visibles en el sistema.

Consejo de defensa

Durante la exposición conviene repetir una idea central: la arquitectura está optimizada para un prototipo IoT defendible, reproducible y entendible. Eso explica tanto sus fortalezas como sus límites actuales.

6. Conclusiones

El Reto 3 queda resuelto de forma integral: se documentan tres vistas arquitectónicas complementarias, se evalúa la solución con un ATAM simplificado, se especifica un despliegue reproducible con Docker Compose y monitoreo básico, y se deja estructurado el portafolio GitHub junto con el material de comunicación final.

La arquitectura propuesta es sólida para un entorno académico y de prototipado serio. Mantiene bajo costo, alta claridad táctica y buena capacidad de evolución. Sus principales límites son conocidos y explícitos: ausencia de alta disponibilidad real, dependencia fuerte del gateway central y una estrategia de observabilidad todavía liviana.

Precisamente por hacer visibles esos límites, la entrega resulta más profesional. No intenta vender una solución perfecta, sino una arquitectura coherente, evaluada y defendible, capaz de mostrar decisiones razonadas y una trayectoria clara de crecimiento hacia versiones más robustas.

Cierre

Como documento final editable, este archivo sirve tanto para entregar el Reto 3 como para ajustar detalles de estilo, nombres de integrantes, capturas o enlaces antes de la presentación oficial.

Anexo A. Archivo docker-compose.yml propuesto

El siguiente listado se entrega como base ejecutable del despliegue. Puede copiarse directamente al repositorio y ajustarse en rutas, credenciales o imágenes específicas del proyecto.

```
version: "3.9"

x-logging: &default-logging
driver: json-file
options:
  max-size: "10m"
  max-file: "3"

services:
  mqtt-broker:
    image: eclipse-mosquitto:2
    restart: unless-stopped
    ports: ["1883:1883"]
    volumes:
      - ./mosquitto/mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
      - mosquitto_data:/mosquitto/data
      - mosquitto_log:/mosquitto/log
    healthcheck:
      test: ["CMD-SHELL", "mosquitto_sub -h localhost -t '$$SYS/broker/version' -C 1 -W 5
>/dev/null 2>&1 || exit 1"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 15s
    logging: *default-logging
    networks: [iot_net]

  postgres:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
      POSTGRES_DB: iot_security
      POSTGRES_USER: iot_user
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
      TZ: ${TZ}
    ports: ["5432:5432"]
    volumes:
      - postgres_data:/var/lib/postgresql/data
      - ./db/init.sql:/docker-entrypoint-initdb.d/init.sql:ro
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U iot_user -d iot_security"]
      interval: 20s
      timeout: 5s
      retries: 5
      start_period: 10s
    logging: *default-logging
    networks: [iot_net]

  node-red:
    image: nodered/node-red:3.1
    restart: unless-stopped
    depends_on:
      mqtt-broker:
        condition: service_healthy
      postgres:
        condition: service_healthy
    environment:
      TZ: ${TZ}
      MQTT_HOST: mqtt-broker
      MQTT_PORT: 1883
      MQTT_USER: ${MQTT_USER}
      MQTT_PASSWORD: ${MQTT_PASSWORD}
      POSTGRES_HOST: postgres
      POSTGRES_DB: iot_security
      POSTGRES_USER: iot_user
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
```

```

    ALERT_THRESHOLD: ${ALERT_THRESHOLD}
ports: ["1880:1880"]
volumes:
  - nodered_data:/data
  - ./nodered/flows.json:/data/flows.json
healthcheck:
  test: ["CMD-SHELL", "wget -qO- http://localhost:1880/healthz >/dev/null 2>&1 || exit 1"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 25s
logging: *default-logging
networks: [iot_net]

dashboard-api:
  build:
    context: ./dashboard
  restart: unless-stopped
  depends_on:
    node-red:
      condition: service_healthy
    postgres:
      condition: service_healthy
  environment:
    TZ: ${TZ}
    APP_PORT: 8000
    POSTGRES_HOST: postgres
    POSTGRES_DB: iot_security
    POSTGRES_USER: iot_user
    POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
  ports: ["8000:8000"]
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:8000/health >/dev/null 2>&1 || exit 1"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 20s
  logging: *default-logging
  networks: [iot_net]

uptime-kuma:
  image: louislam/uptime-kuma:1
  restart: unless-stopped
  ports: ["3001:3001"]
  volumes:
    - kuma_data:/app/data
  healthcheck:
    test: ["CMD-SHELL", "wget -qO- http://localhost:3001 >/dev/null 2>&1 || exit 1"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 20s
  logging: *default-logging
  networks: [iot_net]

sensor-simulator:
  build:
    context: ./simulator
  restart: unless-stopped
  depends on:
    mqtt-broker:
      condition: service_healthy
    node-red:
      condition: service_healthy
  environment:
    TZ: ${TZ}
    MQTT_HOST: mqtt-broker
    MQTT_PORT: 1883
    MQTT_USER: ${MQTT_USER}
    MQTT_PASSWORD: ${MQTT_PASSWORD}
    DEFAULT_ZONE: zonal
    INTERVAL_SECONDS: 8
  command: ["python", "publisher.py", "--mode", "loop"]

```

```
    logging: *default-logging
    networks: [iot_net]

networks:
  iot_net:
    driver: bridge

volumes:
  mosquitto_data:
  mosquitto_log:
  nodered_data:
  postgres_data:
  kuma_data:
```

Anexo B. Variables de entorno, configuración y health endpoints

.env.example

```
POSTGRES_PASSWORD=ChangeMe_123
TZ=America/Bogota
MQTT_USER=edge_zonal
MQTT_PASSWORD=ChangeMe_MQTT
ALERT_THRESHOLD=0.70
APP_PORT=8000
```

Configuración mínima de Mosquitto

```
listener 1883
allow_anonymous false
password_file /mosquitto/config/passwords
persistence true
persistence_location /mosquitto/data/
log_dest file /mosquitto/log/mosquitto.log
```

Respuestas esperadas de salud

```
GET /healthz -> Node-RED
{
  "service": "node-red",
  "status": "ok",
  "mqtt": "connected",
  "database": "reachable",
  "lastHeartbeatSeconds": 12
}

GET /health -> Dashboard/API
{
  "service": "dashboard-api",
  "status": "ok",
  "database": "reachable",
  "version": "1.0.0"
}
```